Advanced Computer Networks

Network Monitoring

Lin Wang

Fall 2021, Period 2

VU VRIJE UNIVERSITEIT AMSTERDAM

Course outline

Warm-up

- Introduction (history, principles)
- Networking basics
- Networking data structures and algorithms
- Network transport

Data centers

- Data center networking
- Data center transport

Programmability

- Software defined networking
- Network automation
- Network function virtualization
- Programmable data plane

Application

- **Network monitoring**
- In-network computing
- Machine learning for networking

Learning objectives

How can we detect “heavy hitters” completely in the data plane?

How can we support different monitoring tasks with a unified data plane data structure?

How can we detect “heavy hitters” completely in the data plane?

Monitoring tasks

Network monitoring is fundamental in network performance optimization and security

Traffic engineering

Flow size distribution

Anomaly detection (DDoS)

Entropy, traffic changes

Worm detection

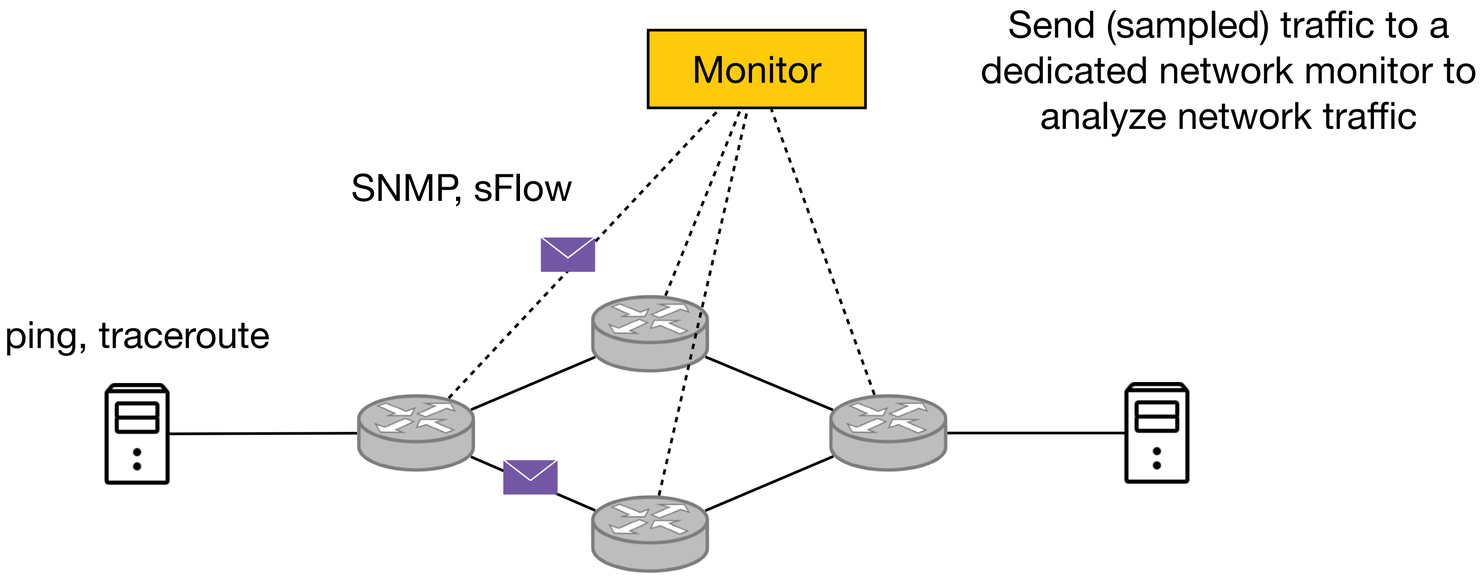
Superspreaders

Accounting

Heavy hitters

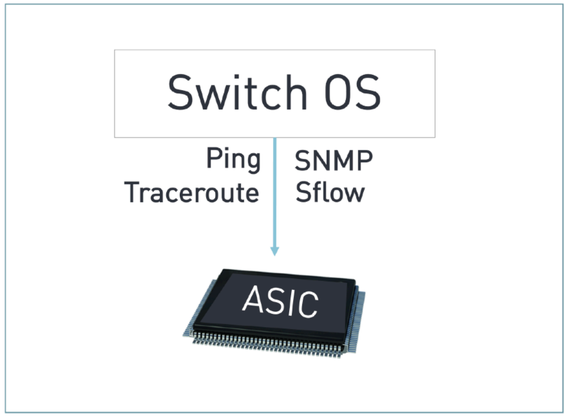
Network monitoring

The use of a system to constantly monitor a computer network for slow or failing components and that notifies the network admins in case of outages or other trouble

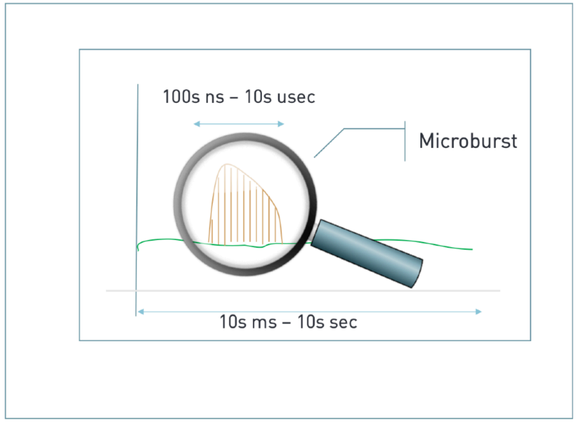


Use data plane network management tools like ping/traceroute to diagnose network issues

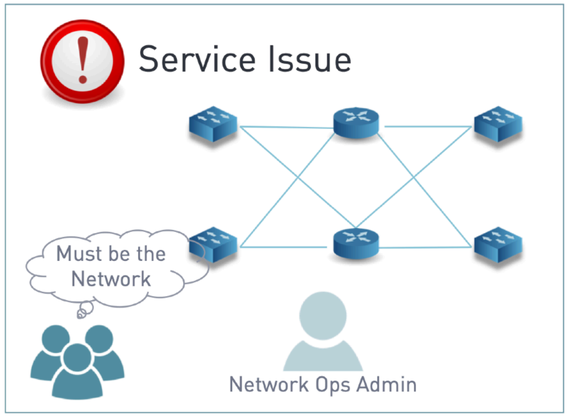
Today's network monitoring problems



Expensive and inefficient



Cannot capture microbursts



No visibility = no control

Network monitoring tasks

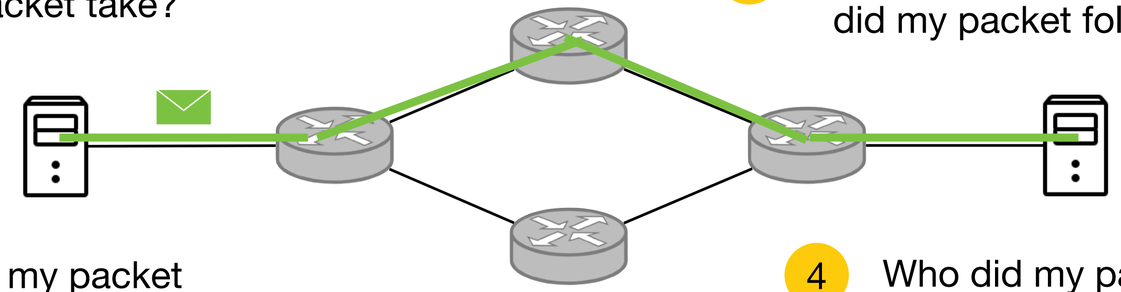
What do we want to know from the network?

I visited switch 1 @720ns, switch 7 @1.8us

In switch 1, I followed rules 39 and 102. In switch 9…

1 Which path did my packet take?

2 Which rules on the switch did my packet follow?



3 How long did my packet queue at each switch?

4 Who did my packet share the queue with?

Delay: 100ns, 300ns, 10200ns…

Flow 1: src1->dst1,
Flow 2: src2->dst2…

How can we obtain such per-packet information in real time?

In-band network telemetry (INT)

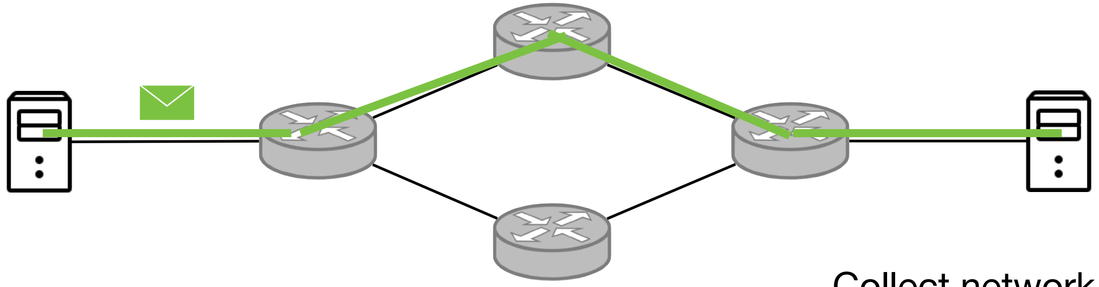
Leverage the programmability of switches to insert monitoring information in the packet header along the network path

```
/* INT: add switch id */
action int_set_header_0() {
  add_header(int_switch_id_header);
  modify_field(int_switch_id_header.switch_id,
               global_config_metadata.switch_id);
}

/* INT: add ingress timestamp */
action int_set_header_1() {
  add_header(int_ingress_tstamp_header);
  modify_field(int_ingress_tstamp_header.ingress_tstamp,
i2e_metadata.ingress_tstamp);
}

/* INT: add egress timestamp */
action int_set_header_2() {
  add_header(int_egress_tstamp_header);
  modify_field(int_egress_tstamp_header.egress_tstamp,
               eg_intr_md_from_parser_aux.egress_global_tstamp);
}
```

Use P4 to implement logic on switches to insert the switch ID, the ingress timestamp, the egress time stamp, and queue information in the packet header.



Collect network monitoring information from the packet header at the receiver side

How can we detect “heavy hitters” completely in the data plane?

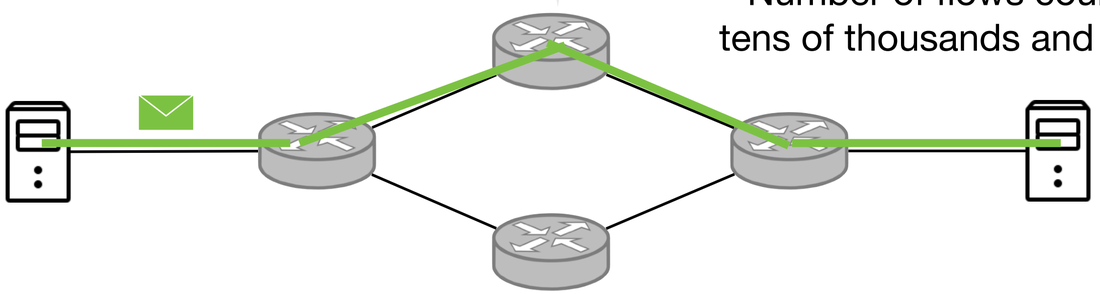
Heavy hitters

Network flows that are larger (in number of packets or bytes) than a fraction t of the total packets seen on the link or the top k flows by size

Flow is defined by combinations of packet header fields, e.g., 5-tuple (src_ip, dst_ip, src_port, dst_port, protocol).

Flow-1: 1750
Flow-2: 1320
Flow-3: 800
…

Number of flows could be tens of thousands and higher



Challenge: finer-grained flows → larger size and number of keys → more bits to represent the key and more entries to track

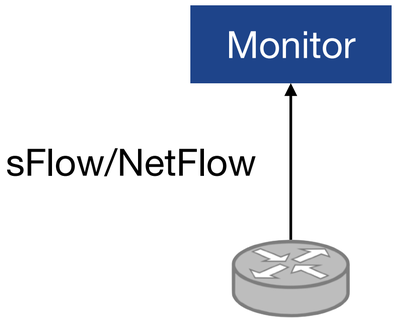
Design goals and constraints

Assume all functionalities need to be implemented in the switch data plane, not involving the control plane

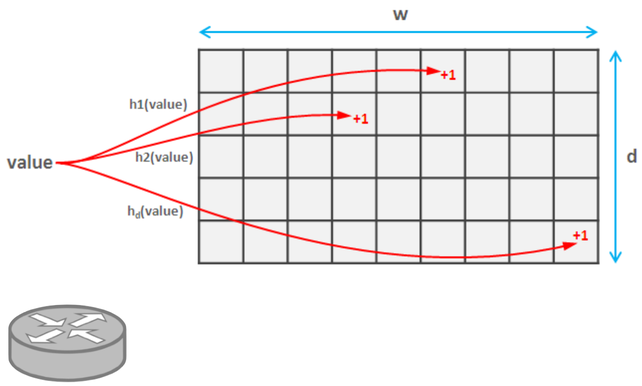
Accuracy: false positives (reporting a non-heavy flow as heavy), false negatives (not reporting a heavy flow), error in estimating the sizes of heavy flows

Overhead: total amount of memory for the data structure, the number of matching stages uses in the switch pipeline

Existing solutions



Packet sampling: use aggressively flow sampling rage (1% or 0.01%) → low accuracy



Streaming algorithms: use count-min / count sketches → does not track flow entities

Can we simply use O(k) counters?

Assume we aim to obtain the top-k heavy flows

	Top-5	
Actual count, tracked	Flow-8	122
	Flow-1	94
	Flow-7	73
	Flow-2	69
	Flow-4	47
Actual count, not tracked	Flow-9	46
	Flow-3	31

2 packets from Flow-9 arrive →

Top-5	
Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-4	47
Flow-9	48
Flow-3	31

Flow-9 should be in top-5 instead of Flow-4

The space-saving algorithm

A counter-based algorithm that uses O(k) counters to track k heavy flows

Actual count, tracked

Top-5

Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-4	47

1 packet from Flow-7 arrives →

Flow-8	122
Flow-1	94
Flow-7	**73+1**
Flow-2	69
Flow-4	47

1 packet from **Flow-9** arrives →

Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-9	**47+1**

Properties of the space-saving algorithm

Suppose the true count of flow key_j is c_j and its count in the table is val_j

Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-4	47

Property 1: no flow counter in the table is ever underestimated, i.e., c_j <= val_j

Property 2: the minimum value in the table val_r is an upper bound on the overestimation error of any counter, e.g., val_j <= c_j + val_r.

Property 3: any flow with true count higher than the average table count, i.e., c_j >= C/m will always be present in the table (C is the total packet count added into the table)

Ahmed Metwally, Divyakant Agrawal, Amr El Abbadi. Efficient computation of frequent and top-k elements in data streams. International Conference on Database Theory (ICDT), 2005.

Properties of the space-saving algorithm

Suppose the true count of flow key_j is c_j and its count in the table is val_j

Property 2: the minimum value in the table val_r is an upper bound on the overestimation error of any counter, e.g., val_j <= c_j + val_r.

Flow-4 →

Flow-8	122	
Flow-1	94	
Flow-7	73	
Flow-2	69	
Flow-3	47	val_r0

Flow-8	122	
Flow-1	94	
Flow-7	73	
Flow-2	69	
Flow-4	**48**	

Flow-8	122	
Flow-1	94	
Flow-4	**88**	
Flow-7	75	
Flow-2	72	val_r

val_j - c_j <= val_r0 <= val_r

Challenges in implementing the space-saving algorithm on switches

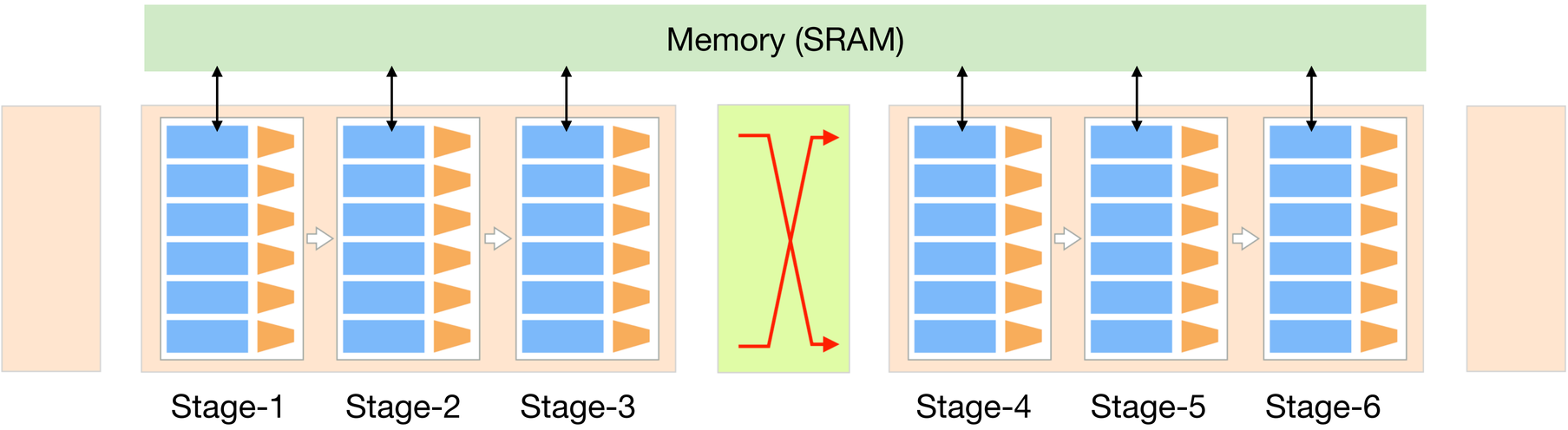
Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-4	47

If the flow has appeared in the table: hash to the flow key and increment the corresponding counter.

If the flow is not contained in the table: find the minimum counter in the table, replace the key with the current flow key, and increment the counter

How to find the minimum counter in the table?

Recall the PISA pipeline



Match-action tables are arranged in stages, where only one memory access is allowed in each stage.

Challenges in implementing the space-saving algorithm on switches

Flow-8	122
Flow-1	94
Flow-7	73
Flow-2	69
Flow-4	47

If the flow has appeared in the table: hash to the flow key and increment the corresponding counter.

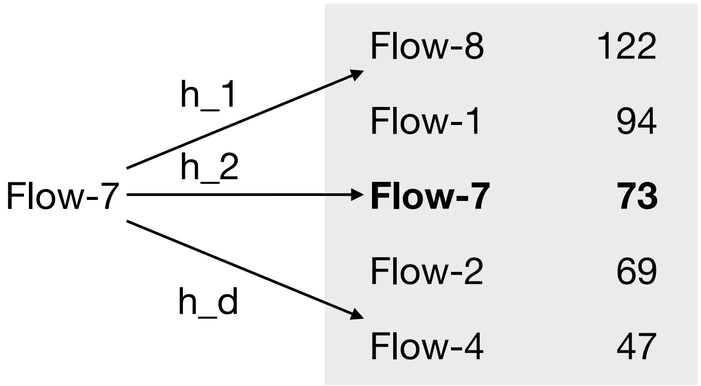
If the flow is not contained in the table: find the minimum counter in the table, replace the key with the current flow key, and increment the counter

Sorted linked list or priority queue → hard to maintain on switches

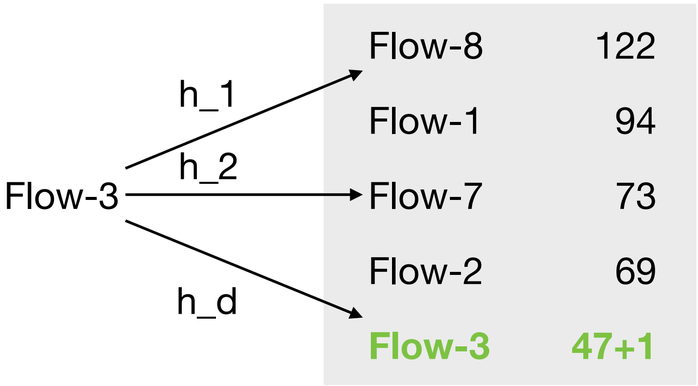
Read k locations, and write back to one location → multiple memory access

Optimization with sampling

Instead of reading all k positions, sample at d positions where d << k



If the flow key appears in one of the hashed locations, increment the corresponding counter.

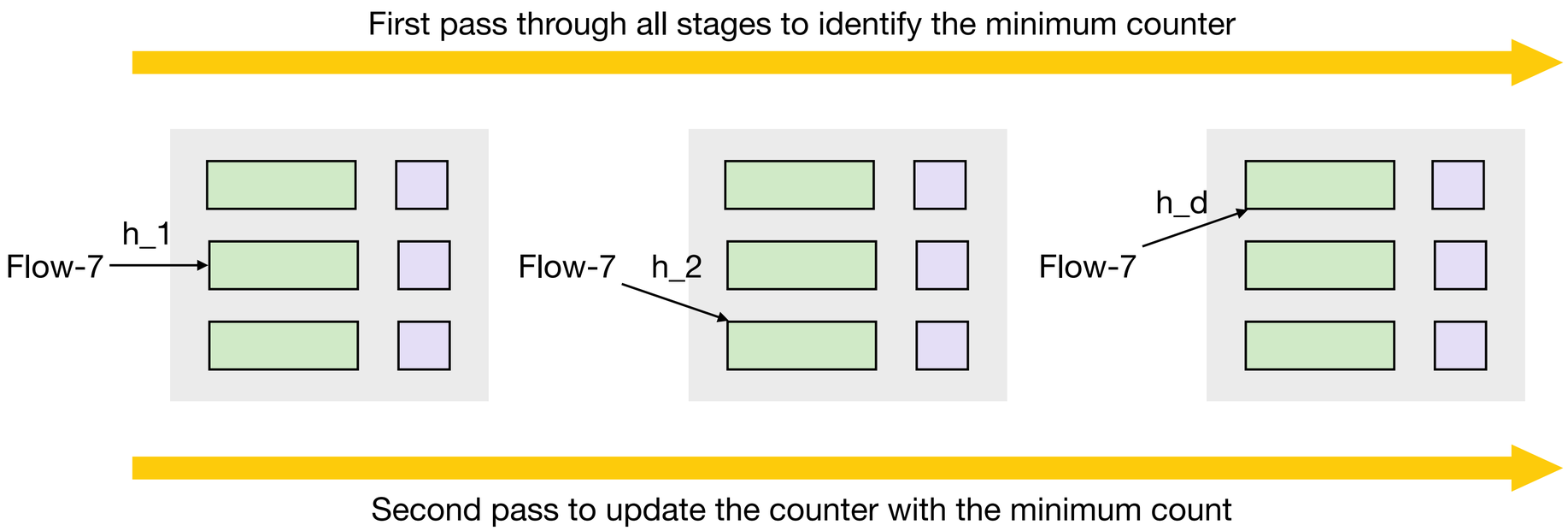


Otherwise, choose the smallest counter among the d positions, and replace the key and increment the counter.

Number of memory reads: d, number of memory writes: 1

Optimization with multi-stages

Split the counter table into d stages and read only once per stage



Second pass → packet recirculation for every packet → the bandwidth is halved

Heavy-Hitter Detection Entirely in the Data Plane

Vibhaalakshmi Sivaraman
Princeton University

Srinivas Narayana
MIT CSAIL

Ori Rottenstreich
Princeton University

S. Muthukrishnan
Rutgers University

Jennifer Rexford
Princeton University

ABSTRACT

Identifying the "heavy hitter" flows or flows with large traffic volumes in the data plane is important for several applications *e.g.,* flow-size aware routing, DoS detection, and traffic engineering. However, measurement in the data plane is constrained by the need for line-rate processing (at 10-100Gb/s) and limited memory in switching hardware. We propose HashPipe, a heavy hitter detection algorithm using emerging programmable data planes. HashPipe implements a pipeline of hash tables which retain counters for heavy flows while evicting lighter flows over time. We prototype HashPipe in P4 and evaluate it with packet traces from an ISP backbone link and a data center. On the ISP trace (which contains over 400,000 flows), we find that HashPipe identifies 95% of the 300 heaviest flows with less than 80KB of memory.

variations [1, 5]) can enable dynamic routing of heavy flows [16, 35] and dynamic flow scheduling [41].

It is desirable to run heavy-hitter monitoring at all switches in the network all the time, to respond quickly to short-term traffic variations. *Can packets belonging to heavy flows be identified as the packets are processed in the switch*, so that switches may treat them specially?

Existing approaches to monitoring heavy items make it hard to achieve reasonable accuracy at acceptable overheads (§2.2). While packet *sampling* in the form of NetFlow [12] is widely deployed, the CPU and bandwidth overheads of processing sampled packets in software make it infeasible to sample at sufficiently high rates (sampling just 1 in 1000 packets is common in practice [34]). An alternative is to use sketches, *e.g.,* [14, 24, 25, 45] that *hash* and *count* all packets in switch hardware. However, these systems incur a large memory overhead to retrieve the heavy hitters — ideally, we wish

ACM SOSR 2017

Feed-forward packet processing

Two key ideas: tracking a rolling minimum and always inserting in the first stage

New key K

h_1 (K) -> K1

Stage 1	
A	5
K1	4
B	6
C	10

Stage 2	
K2	3
D	15
E	25
F	100

Stage 3	
G	4
K3	3
H	10
I	9

First stage: if key k is a match, increment the counter and finish processing; otherwise, always insert the new key with count 1 at the hashed location and carry the old one (B here) with the metadata to the next stage

Feed-forward packet processing

Two key ideas: tracking a rolling minimum and always inserting in the first stage

Stage 1	
A	5
K	**1**
B	6
C	10

(K1, 4)

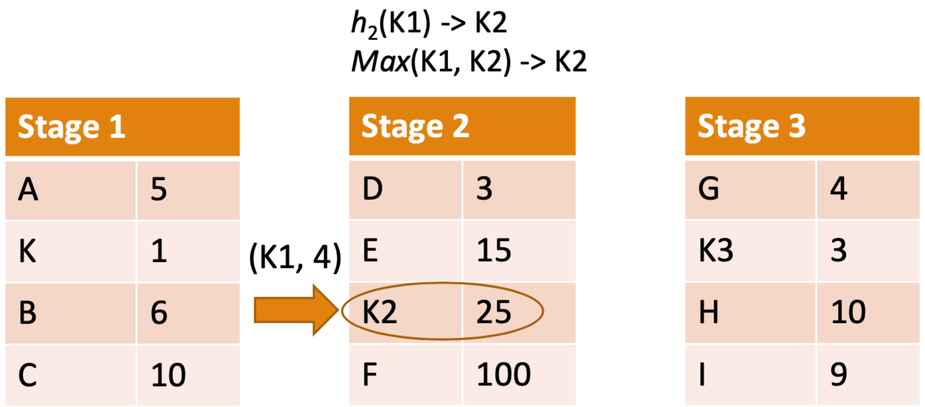
Stage 2	
D	3
E	15
K2	25
F	100

Stage 3	
G	4
K3	3
H	10
I	9

Later stages: compare the counter at the hashed position (with the key from the metadata) and the counter from the metadata, replace the key-counter in the table if the one carried in the metadata is larger

Feed-forward packet processing

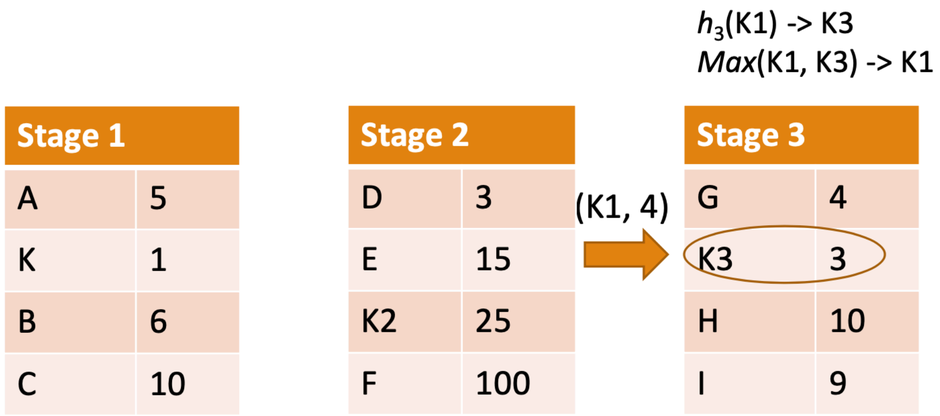
Two key ideas: tracking a rolling minimum and always inserting in the first stage



Later stages: compare the counter at the hashed position (with the key from the metadata) and the counter from the metadata, replace the key-counter in the table if the one carried in the metadata is larger

Feed-forward packet processing

Two key ideas: tracking a rolling minimum and always inserting in the first stage



Later stages: compare the counter at the hashed position (with the key from the metadata) and the counter from the metadata, replace the key-counter in the table if the one carried in the metadata is larger

Feed-forward packet processing

Two key ideas: tracking a rolling minimum and always inserting in the first stage

Stage 1	
A	5
K	1
B	6
C	10

Stage 2	
D	3
E	15
K2	25
F	100

Stage 3	
G	4
K1	**4**
H	10
I	9

Last stages: evict a relatively small flow

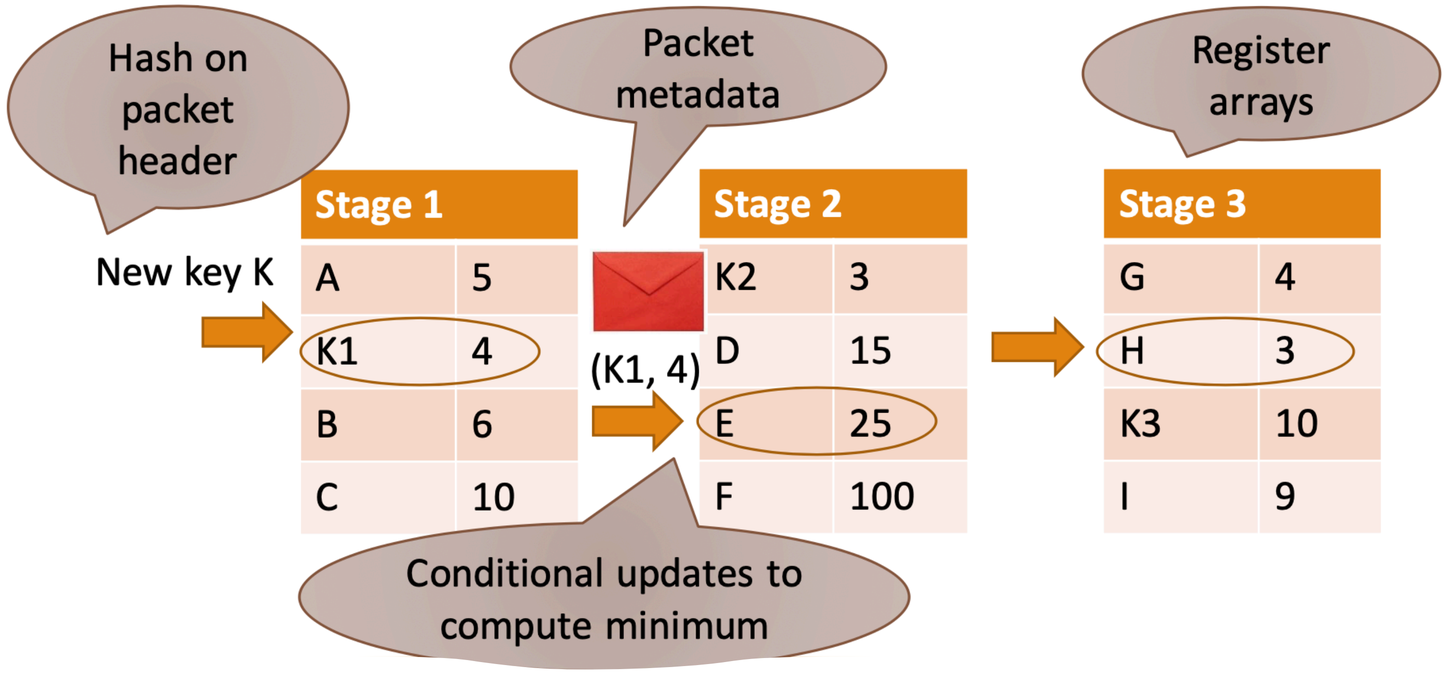
HashPipe summary

Split hash table into d stages

Condition	Stage 1	Stage 2-d
Empty	Insert with value 1	Insert key and value carried
Match	Increment value by 1	Coalesce value carried with value in table
Mismatch	Insert new key with value, evict and carry key in table	Keep key with higher value and carry the other

HashPipe implemented in P4

All required functionalities are supported in P4



How can we support different monitoring tasks with a unified data structure in the data plane?

Monitoring tasks

Network monitoring is fundamental in network performance optimization and security

Traffic engineering

Flow size distribution

Anomaly detection (DDoS)

Entropy, traffic changes

Worm detection

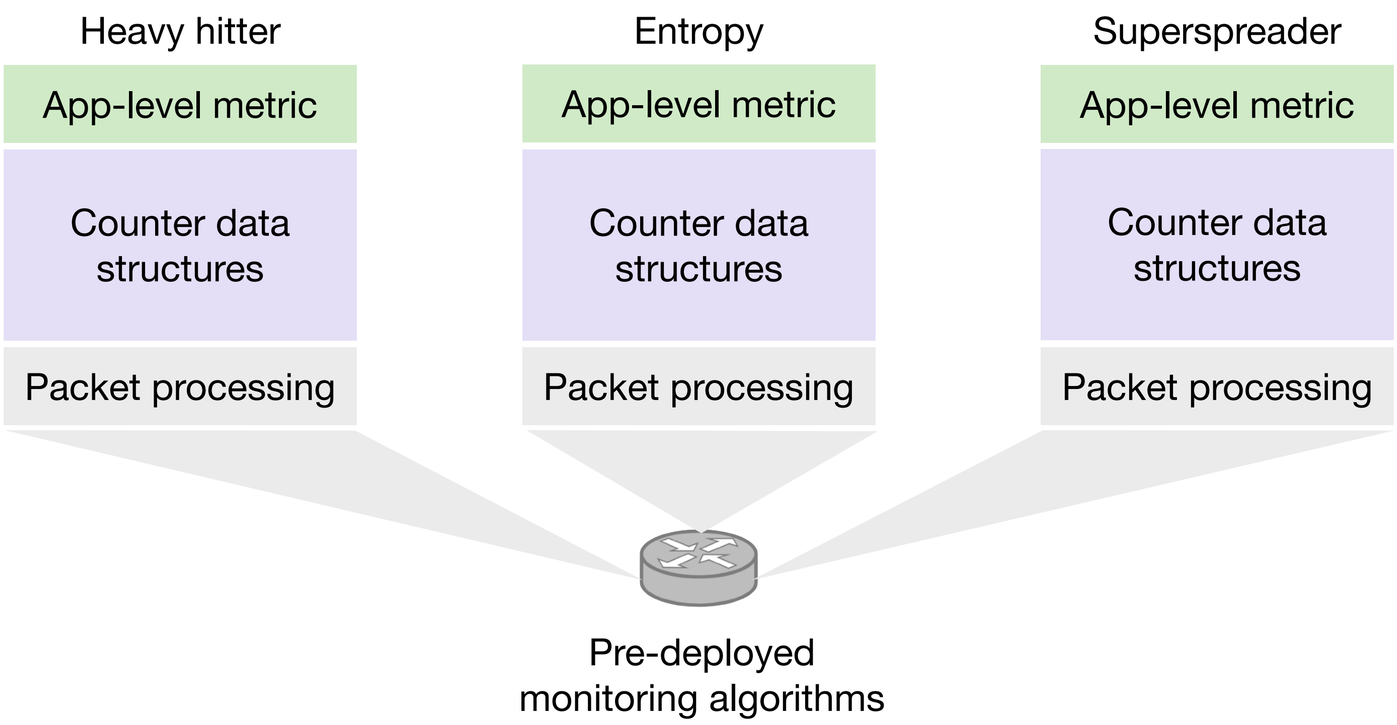
Superspreaders

Accounting

Heavy hitters

Task-specific data structures

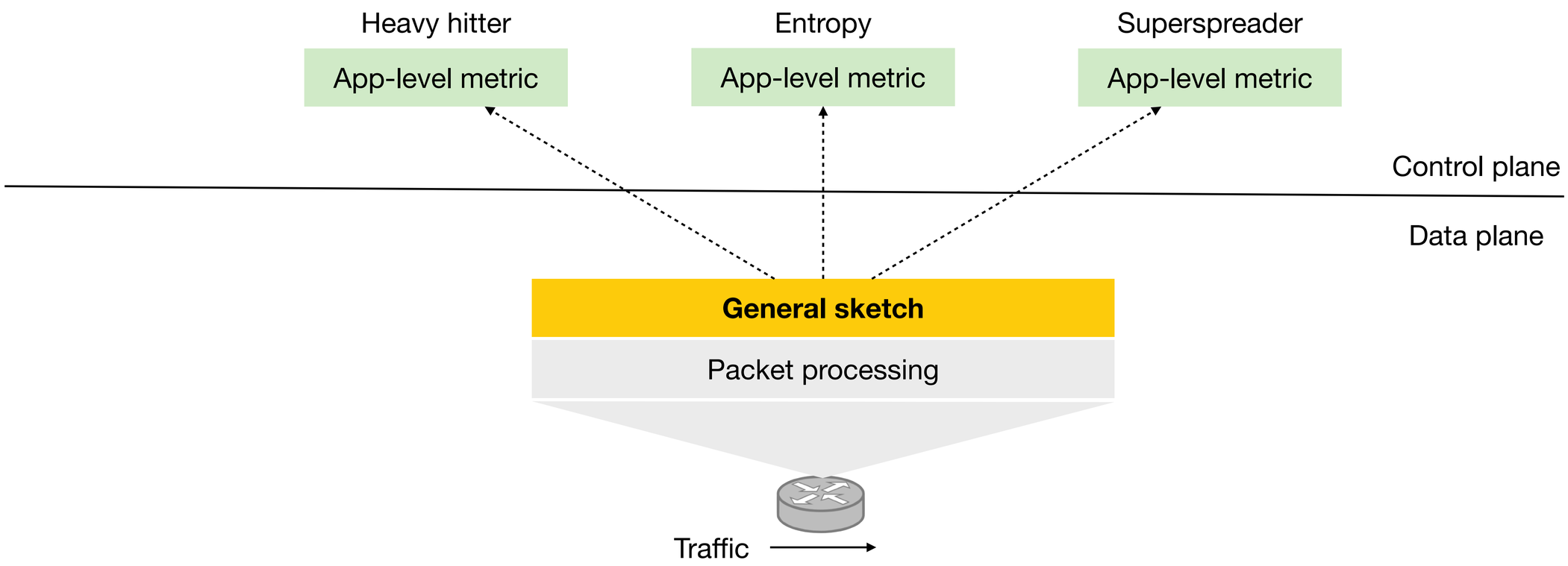
For different monitoring tasks, different data structures are developed in the data plane



Tight binding between data plane and control plane, not general!

The ideal case

One sketch to rule them all



Late binding between control plane monitoring tasks and the data plane data structure

Yes, that is possible!

One Sketch to Rule Them All: Rethinking Network Flow Monitoring with UnivMon

Zaoxing Liu[†], Antonis Manousis[⋆], Gregory Vorsanger[†], Vyas Sekar[⋆], Vladimir Braverman[†]
[†] Johns Hopkins University [⋆] Carnegie Mellon University

ABSTRACT

Network management requires accurate estimates of metrics for many applications including traffic engineering (e.g., heavy hitters), anomaly detection (e.g., entropy of source addresses), and security (e.g., DDoS detection). Obtaining accurate estimates given router CPU and memory constraints is a challenging problem. Existing approaches fall in one of two undesirable extremes: (1) low fidelity general-purpose approaches such as sampling, or (2) high fidelity but complex algorithms customized to specific application-level metrics. Ideally, a solution should be both general (i.e., supports many applications) and provide accuracy comparable to custom algorithms. This paper presents *UnivMon*, a framework for flow monitoring which leverages recent theoretical advances and demonstrates that it is possible to achieve both generality and high accuracy. UnivMon uses an application-agnostic data plane monitoring primitive; different (and possibly unforeseen) estimation algorithms run

1 Introduction

Network management is multi-faceted and encompasses a range of tasks including traffic engineering [11, 32], attack and anomaly detection [49], and forensic analysis [46]. Each such management task requires accurate and timely statistics on different application-level metrics of interest; e.g., the flow size distribution [37], heavy hitters [10], entropy measures [38, 50], or detecting changes in traffic patterns [44].

At a high level, there are two classes of techniques to estimate these metrics of interest. The first class of approaches relies on *generic flow monitoring*, typically with some form of packet sampling (e.g., NetFlow [25]). While generic flow monitoring is good for coarse-grained visibility, prior work has shown that it provides low accuracy for more fine-grained metrics [30, 31, 43]. These well-known limitations of sampling motivated an alternative class of techniques based on *sketching* or *streaming* algorithms. Here, custom online algorithms and data structures are designed for specific met-

ACM SIGCOMM 2016

Universal streaming

Basic streaming algorithms: assume a stream of length m with n unique items

1	1	5	1	3	3	1	2	4	6	5

...

Frequency vector: $< f_1, f_2, ..., f_n >$

Frequency moments: $F_k = \sum_{i=1}^{n} f_i^k$

F_2 can be obtained with Count sketch or AMS sketch

One algorithm solves one problem

Universal streaming

1	1	5	1	3	3	1	2	4	6	5

...

Frequency vector: $< f_1, f_2, ..., f_n >$

Can we compute an arbitrary function with a single sketch

$$\text{G-sum} = \sum_{i=1}^{n} g(f_i)$$

Theory of universal streaming

Theorem 1: There exists a universal approach to estimate G-sum when function g(*) is non-decreasing such that g(0) = 0 and g(x) does not monotonically grow faster than x^2.

Theorem 2: A universal sketch construction can be used to estimate G-sum with high probability using poly-logarithmic memory.

Intuition of universal sketch

Informal definition: Item i is a g-heavy hitter if changing its frequency f_i significantly affects its G-sum.

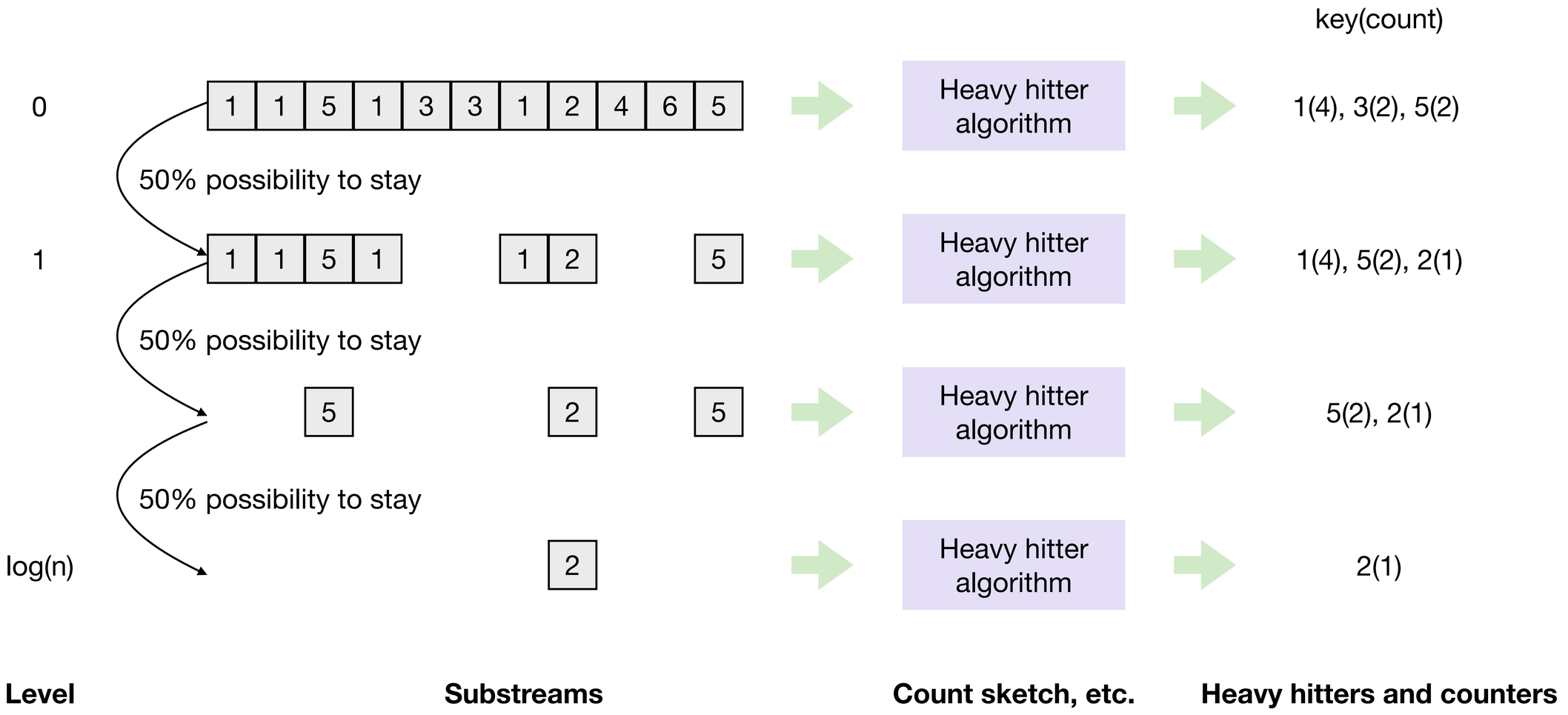
Case 1: There is one sufficiently large g-heavy hitter. Most of mass is concentrated at this heavy hitter. Use L2 heavy hitter algorithm to find such a heavy hitter.

$$\text{L2 norm} = (\sum_{i=1}^{n} f_i^2)^{1/2}$$

Case 2: There is NO single sufficiently large g-heavy hitter. Approximate with L2 heavy hitters on a series of sampled substreams of increasingly smaller size.

Vladimir Braverman, Rafail Ostrovsky. Zero-one frequency laws. ACM STOC, 2010.

Universal sketch data structure



From universal sketch counters to G-sum

Algorithm 2 UnivMon Offline Estimation Algorithm

Input: Set of heavy hitters $Q = \{Q_0, \ldots, Q_{\log(n)}\}$

1. For $j = 0 \ldots \log(n)$, call g() on all counters $w_j(i)$ in Q_j. After $g()$, the i-th entry in Q_j is $g(w_j(i))$.
2. Compute $Y_{\log(n)} = \sum_i g(w_{\log(n)}(i))$.
3. For each j from $\log(n) - 1$ to 0, compute:

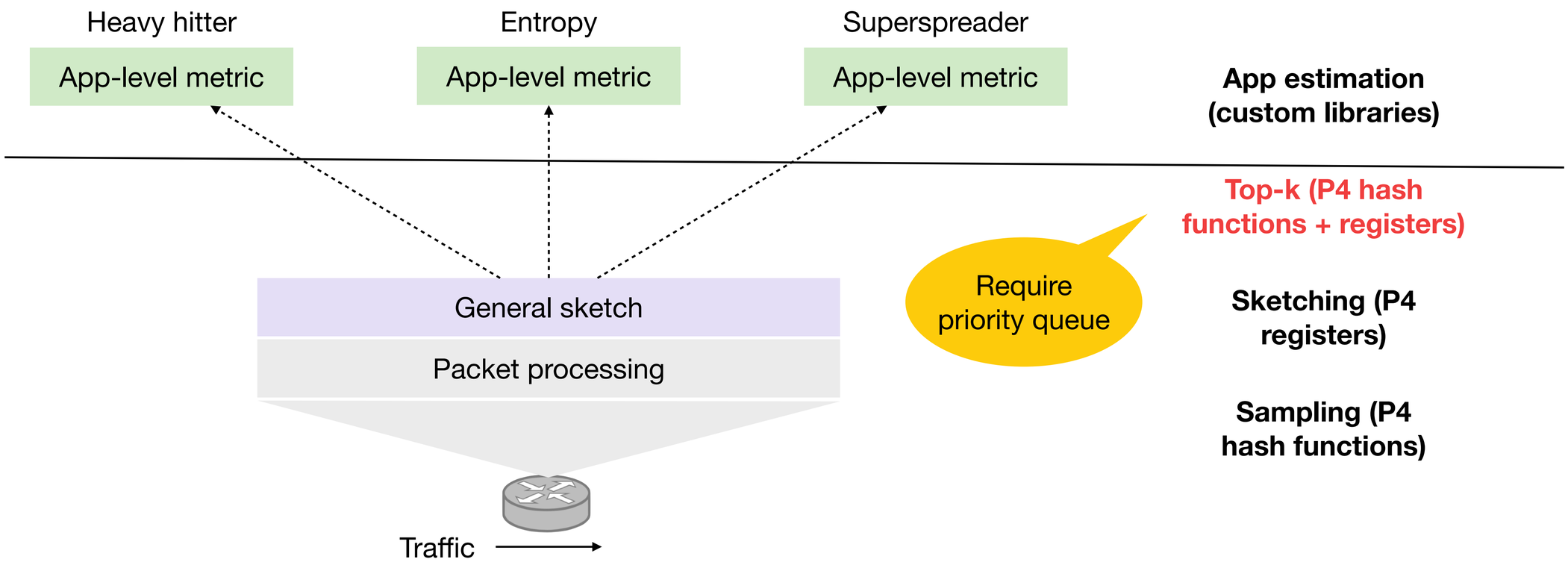
$$Y_j = 2Y_{j+1} + \sum_{i \in Q_j} (1\text{-}2h_{j+1}(i))\ g(w_j(i))$$

Output: Y_0

Can be applied to network monitoring tasks: heavy hitter, DDoS victim detection, change detection, entropy estimation, global iceberg detection

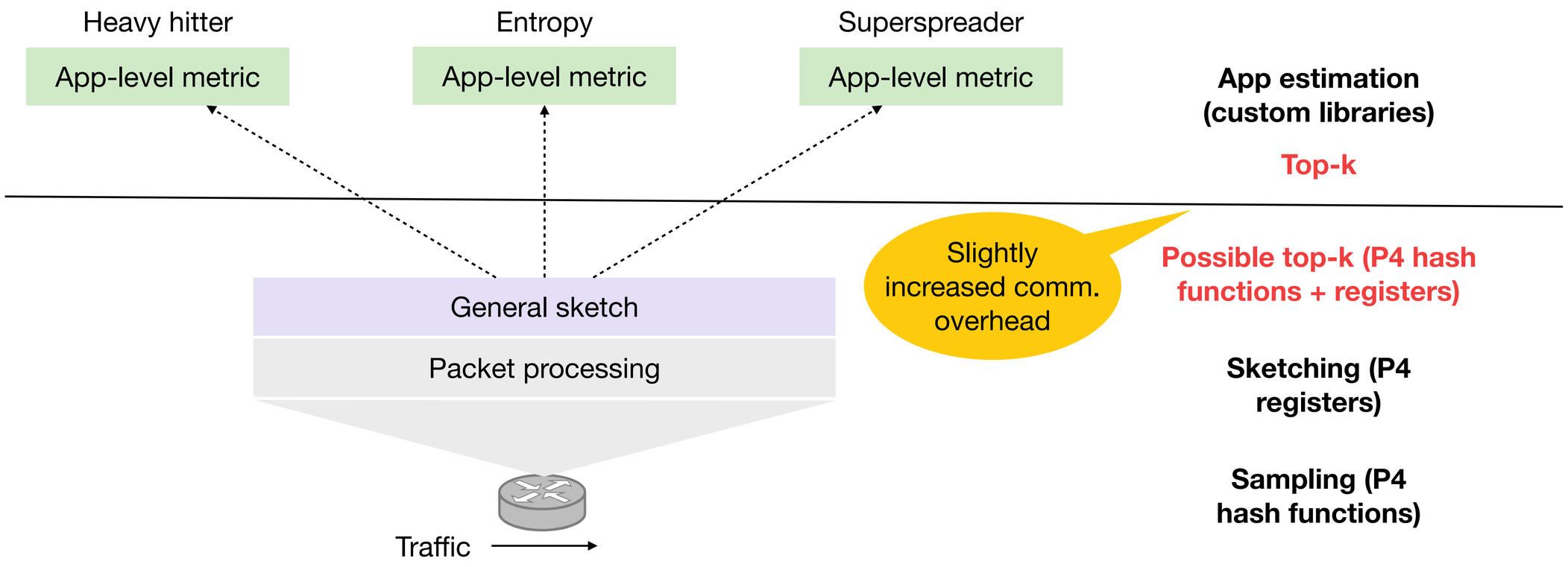
UnivMon implementation

Where to implement which functionality?

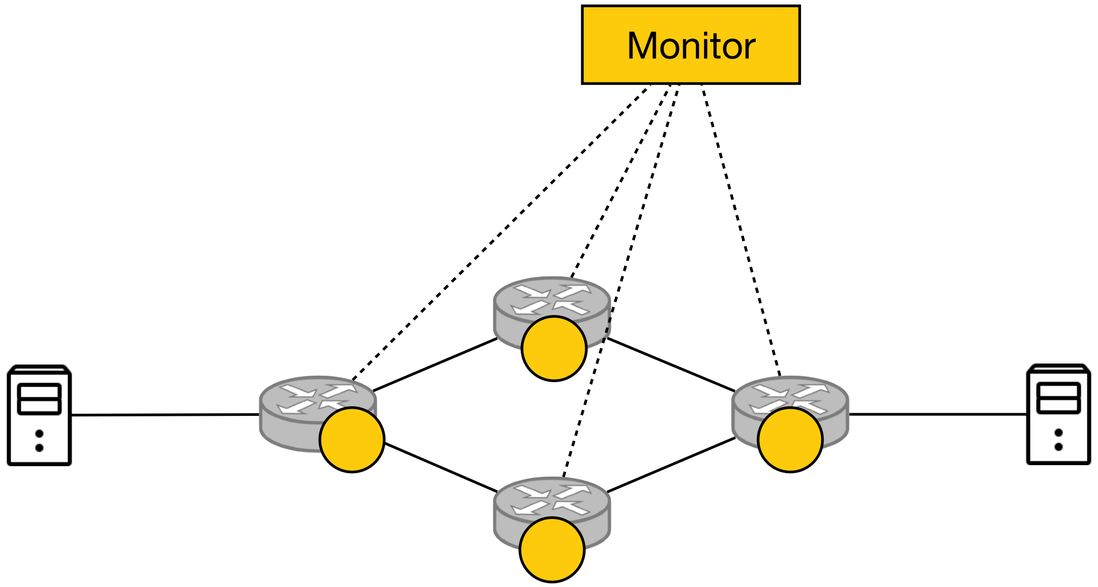


UnivMon implementation

Where to implement which functionality?

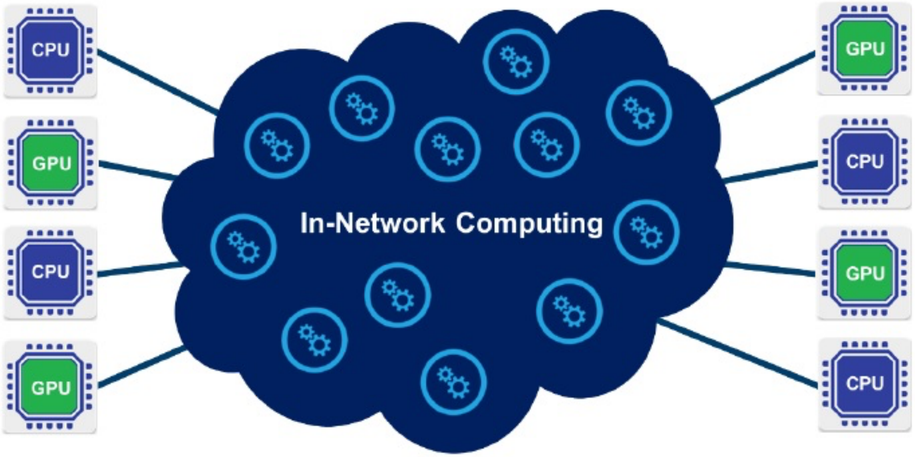


Summary



Implementing heavy hitter detection completely in the data plane and support various monitoring tasks with a unified sketch data structure in the data plane.

Next time: in-network computing



When the network becomes the computer!